EXPERIMENT – 3

Title: TCP Server Client

Objective:

- To study TCP based client–server communication.
- To implement a TCP server and a TCP client using Python.
- To understand reliable data transfer using sockets

Software / Tools Required

- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

Theory

TCP (Transmission Control Protocol) is a reliable and connection-oriented protocol used for data communication over a network. Before any data transfer takes place, a connection is established between the client and the server. TCP ensures that data is delivered correctly and in order.

In a client–server model, the server waits for incoming client requests on a fixed IP address and port number. The client initiates the connection and receives data from the server. Acknowledgments are exchanged to confirm successful data reception. The connection continues until either the client or server requests termination.

In this experiment, the TCP server sends random data to the client. The client receives the data and sends an acknowledgment back to the server. This process continues until the client closes the connection.

Diagram (TCP Communication):

```
Process 1 (Client)                    Process 2 (Server)
------------------                   --------------------
        |                                     |
        |  S1 = (IP1, PORT1)                  |  S2 = (IP2, PORT2)
        |                                     |
        |-------- TCP Connection ------------>|
        |                                     |
       PC1                                   PC2
```

Procedure / Algorithm

1. Create a TCP socket for the server using the netpack module.
2. Bind the server socket to a local IP address and port number.
3. Start the server and wait for a client connection.
4. Create a TCP socket on the client side.
5. Connect the client socket to the server's IP address and port.
6. After connection, the server sends random data to the client.
7. The client receives the data and displays it on the terminal.
8. The client sends an acknowledgment message to the server.
9. Steps 6 to 8 repeat until the client chooses to close the connection.
10. Close the client and server sockets properly.

Program Code

TCP Server Program:

```python
import netpack as npk
import random

PORT=9999
server_ip="127.0.0.1"
ADDR=(server_ip, PORT)


# Step-1
server_socket=npk.create_tcp_socket()

# Step-2
npk.start_tcp_server(server_socket, ADDR)

# Step-3
print("[WAIT] Server waits for client")
(client_socket, addr)=npk.accept_tcp_client(server_socket)
print('[CONNECTED] Client with IP : {} & PORT : {}'.format(addr[0], addr[1]))

while True:
    val=random.randint(0,100)
    npk.send_data_to_tcp_client(client_socket,val)
    print("[SEND] Data {}".format(val))

    data=npk.read_data_from_tcp_client(client_socket)
    if data=="close":
        npk.close_socket_connection(client_socket)
        print("[=] TCP Client requesting to close connection")
        break
    else:
        print("[RECEV] {}".format(data))

npk.close_socket_connection(server_socket)
```

TCP Client Program:

```python
import netpack as npk

PORT=9999
client_ip="127.0.0.1"

server_ip=client_ip
ADDR=(server_ip, PORT)


# Step-1
client_socket=npk.create_tcp_socket()

# Step-2
npk.connect_to_tcp_server(client_socket, ADDR)
print('[CONNECTED] Server with IP : {} & PORT : {}'.format(ADDR[0], ADDR[1]))

while True:
    data=npk.read_data_from_tcp_server(client_socket)
    print('[=] Message from server : {}'.format(data))

    flag=input('Close connection y/n : ')
    if flag=='y' or flag=='Y':
        npk.send_data_to_tcp_server(client_socket,"close")
        npk.close_socket_connection(client_socket)
        print("[=] Requested for closing connection")
        break
    else:
        npk.send_data_to_tcp_server(client_socket,"+ACK")
```

Output:

Client Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH
[CONNECTED] Server with IP : 127.0.0.1 & PORT : 9999
[=] Message from server : 3
Close connection y/n : n
[=] Message from server : 60
Close connection y/n : n
[=] Message from server : 90
Close connection y/n : n
[=] Message from server : 42
Close connection y/n : n
[=] Message from server : 31
Close connection y/n :
```

Server Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH
[WAIT] Server waits for client
[CONNECTED] Client with IP : 127.0.0.1 & PORT : 34660
[SEND] Data 3
[RECEV] +ACK
[SEND] Data 60
[RECEV] +ACK
[SEND] Data 90
[RECEV] +ACK
[SEND] Data 42
[RECEV] +ACK
[SEND] Data 31
```

Result

The TCP client and server successfully communicated with each other. The client correctly received all data sent by the server and acknowledgments were exchanged without any error.

Conclusion

This experiment demonstrated TCP based client–server communication using Python. It showed how a reliable connection is established using sockets, how data is exchanged between the server and client, and how acknowledgments ensure successful communication. The experiment helped in understanding basic socket programming concepts used in computer networks.